

simple program that opens an image and displays it on the screen:

```
#include "highgui.h"
int main( int argc, char** argv ) {
    IplImage* img = cvLoadImage( argv[1] );
    cvNamedWindow( "Example1", CV_WINDOW_AUTOSIZE );
    cvShowImage( "Example1", img );
    cvWaitKey(0);
    cvReleaseImage( &img );
    cvDestroyWindow( "Example1" );
}
```

When this code is compiled and executed from the command line with a single argument, the program loads the image into memory and displays it on the screen. Then it waits until the user presses a key, at which time it closes the window and exits.

Reading an AVI video file

Playing a video file with OpenCV is as easy as displaying a single picture:

```
#include "highgui.h"
int main( int argc, char** argv ) {
    cvNamedWindow( "Example2", CV_WINDOW_AUTOSIZE );
    CvCapture* capture = cvCreateFileCapture( argv[1] );
    IplImage* frame;
    while(1) {
        frame = cvQueryFrame( capture );
        if( !frame ) break;
        cvShowImage( "Example2", frame );
        char c = cvWaitKey(33);
        if( c == 27 ) break;
    }
    cvReleaseCapture( &capture );
    cvDestroyWindow( "Example2" );
}
```

Accessing the webcam using OpenCV

Accessing the webcam or camera device of a computer is fairly simple with OpenCV. Simply call *open()* on a *cv::VideoCapture* object (OpenCV's method of accessing your camera device), and pass 0 as the default camera ID number. Some systems have multiple cameras attached or they do not work as default camera 0; so it is common practice to allow users to pass the desired camera number as a command-line argument, in case they want to try camera 1, 2, or -1, for example. We can also try to set the camera resolution to 640 x 480 using *cv::VideoCapture::set()*, in order to run faster on high-resolution cameras.

Depending on the camera model, driver, or system, OpenCV might not change the properties of your camera.

But since this is not important for our project, let's not worry about this aspect.

Put the following code in the *main()* function of your *main_desktop.cpp*:

```
int cameraNumber = 0;
if (argc > 1)
    cameraNumber = atoi(argv[1]);
// Get access to the camera.
cv::VideoCapture camera;
camera.open(cameraNumber);
if (!camera.isOpened()) {
    std::cerr << "ERROR: Could not access the camera or video!"
    <<
    std::endl;
    exit(1);
}
// Try to set the camera resolution.
camera.set(cv::CV_CAP_PROP_FRAME_WIDTH, 640);
camera.set(cv::CV_CAP_PROP_FRAME_HEIGHT, 480);
After the webcam has been initialized, you can grab the current camera image as
a cv::Mat object (OpenCV's image container). You can grab each camera frame
by using the C++ streaming operator from your
cv::VideoCapture object into a
cv::Mat object, just like if you were getting input from a console.
```

OpenCV makes it very easy to load a video file (AVI or MPEG file) and use it instead of a webcam. The only difference to the code would be that you have to create the *cv::VideoCapture* object with the video filename, such as *camera.open("my_video.avi")*, rather than the camera number, such as *camera.open(0)*. Both methods create a *cv::VideoCapture* object that can be used in the same way.

Summary

OpenCV is equipped with lots of algorithms associated with digital image processing as well as computer vision, and such algorithmic execution can be explored from the online documentation easily available. OpenCV has rich documentation in tutorials, user guides, books, quick starts, references and wiki modules. Besides, there is a huge community out there to keep track of the bugs and to troubleshoot. 

By: Dr Gaurav Kumar

The author is the MD of Magma Research and Consultancy Pvt Ltd, Ambala Cantt, Haryana. With 16 years' experience in teaching, in industry and in research, he is a projects contributor for the Web-based source code repository SourceForge.net. He can be contacted at kumargaurav.in@gmail.com